

第 1 章

クォータニオンと回転行列

SO(3) に属する回転行列とクォータニオンには対応関係が存在します。姿勢制御のシミュレーションでよく使う式について整理しておきます。

1.1 クォータニオンの定義

1.1.1 一般的なクォータニオン

クォータニオン（四元数） q は、スカラ部 w とベクトル部 (x, y, z) を用いて以下のように表されます。

$$q = w + xi + yj + zk \quad (\text{または } q = (w, x, y, z))$$

1.1.2 回転行列と回転を表すクォータニオン

任意の単位ベクトル $\mathbf{u} = (u_x, u_y, u_z)$ を回転軸とし、その軸の周りに角度 θ だけ回転させる行列 $\mathbf{R}$ は、ロドリゲスの公式を用いてつぎのように表せます。

$$\mathbf{R} = \mathbf{I} + \sin \theta [\mathbf{u}]_{\times} + (1 - \cos \theta) [\mathbf{u}]_{\times}^2 \quad (1.1)$$

ある3次元ベクトル $\mathbf{v}$ を上記の回転軸と回転角で回転させる操作は回転行列 $\mathbf{R}$ をベクトルに掛けて

$$\mathbf{v}' = \mathbf{R}\mathbf{v} \quad (1.2)$$

となります。右辺の積は行列とベクトルの積です。

これに対応し、回転の作用をもつクォータニオンはつぎのように表されます。

$$\begin{aligned} q &= \cos\left(\frac{\theta}{2}\right) + \mathbf{u} \sin\left(\frac{\theta}{2}\right) \\ &= \left(\cos\frac{\theta}{2}, u_x \sin\frac{\theta}{2}, u_y \sin\frac{\theta}{2}, u_z \sin\frac{\theta}{2}\right) \end{aligned} \quad (1.3)$$

あるベクトル $\mathbf{v}$ をクォータニオンで回転させる操作は (1.3) 式の回転を表すクォータニオン q を用いて

$$\mathbf{v}' = q\mathbf{v}q^* \quad (1.4)$$

となります。ただし、 $\mathbf{v}_q$ はスカラー部が0、ベクトル部が $\mathbf{v}$ の四元数です。 q^* は q の共役四元数で、スカラー部は q のスカラー部と同じで、ベクトル部については $\mathbf{v}$ の符号を反転させたものから構成されます。

1.1.3 クォータニオンの積の定義

(1.4) 式の右辺にはクォータニオンの積によって書かれています。ここにおける「積」はスカラー部とベクトル部の組み合わせつぎのように定義されるものです。例えば2つのクォータニオン $q_1 = (w_1, \mathbf{v}_1)$ と $q_2 = (w_2, \mathbf{v}_2)$ の積であれば、

$$q_1 q_2 = (w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$$

と定義されています。この計算結果もスカラー部とベクトル部の組み合わせで表されるクォータニオンになっています。

1.2 単位行列と恒等クォータニオン

R の特別な場合として、回転角 $\theta = 0$ の場合を考えます。このとき、回転行列 R は単位行列 I となります。

これに対応するクォータニオンは、(1.3) 式において $\theta = 0$ とすることにより、スカラ部が 1、ベクトル部が $(0, 0, 0)$ のクォータニオン

$$q = 1 + 0i + 0j + 0k \quad (1.5)$$

になります。これは「恒等クォータニオン」と呼ばれます。

1.2.1 単位クォータニオンと回転を表すクォータニオンの関係

単位クォータニオンとはノルムが 1 である四元数（クォータニオン）です。クォータニオン

$$q = (w, x, y, z)$$

のノルムはつぎのように計算されます。

$$\|q\| = \sqrt{w^2 + x^2 + y^2 + z^2}$$

回転を表す (1.3) 式のクォータニオンに対して上式のノルムを計算すると 1 になることが確かめられます。したがって、「(1.3) 式の回転を表すクォータニオン」は「単位クォータニオン」になります。

逆に「単位クォータニオン」であれば「(1.3) 式の回転を表すクォータニオン」の形式で表すことができます。任意の単位クォータニオン $q = w + \mathbf{v}$ (ただし $\mathbf{v} = (x, y, z)$) について、ノルムが 1 であることから

$$w^2 + \|\mathbf{v}\|^2 = 1 \quad \implies \quad -1 \leq w \leq 1$$

が成り立ちます。二乗の和が 1 になることから、三角関数の $\cos^2 \phi + \sin^2 \phi = 1$ と比較して、ある角度 ϕ が存在し、

$$w = \cos \phi, \quad \|\mathbf{v}\| = \sin \phi$$

と置くことができます。 $\|\mathbf{v}\| = \sin \phi$ である場合、単位ベクトル $\mathbf{u} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$ を用いて、ベクトル部 $\mathbf{v}$ を

$$\mathbf{v} = \mathbf{u} \sin \phi$$

と表すことができます。したがって、単位クォータニオン q は

$$q = \cos \phi + \mathbf{u} \sin \phi$$

の形式で表すことができ、これは (1.3) 式の回転を表すクォータニオンの形式と同じです。

1.2.2 静止座標系と機体座標系の変換

回転行列 $\mathbf{R}$ は、機体座標系での成分で表されたベクトル $\mathbf{v}_b$ を静止座標系の成分で表されたベクトル $\mathbf{v}_a$ に変換するために使われることがあります。その変換式は

$$\mathbf{v}_a = \mathbf{R} \mathbf{v}_b \quad (1.6)$$

です。逆の場合は

$$\mathbf{v}_b = \mathbf{R}^T \mathbf{v}_a \quad (1.7)$$

です。

クォータニオン q も同様に、静止座標系での成分で表されたベクトル $\mathbf{v}_a$ を機体座標系での成分で表されたベクトル $\mathbf{v}_b$ に変換するために使われることがあります。その変換式は

$$\mathbf{v}_{qa} = q \mathbf{v}_{qb} q^* \quad (1.8)$$

です。逆の場合は

$$\mathbf{v}_{qb} = q^* \mathbf{v}_{qa} q \quad (1.9)$$

です。ただし、 $\mathbf{v}_{qa}$ はスカラー部が 0、ベクトル部が $\mathbf{v}_a$ の四元数であり、 $\mathbf{v}_{qb}$ はスカラー部が 0、ベクトル部が $\mathbf{v}_b$ の四元数です。

1.3 角速度ベクトルと回転の変化率の関係

角速度ベクトル ω を機体座標系で表した場合、回転行列 R の時間微分 $\dot{R}$ は次のように表されます。

$$\dot{R} = R[\omega]_{\times} \quad (1.10)$$

ここで、 $[\omega]_{\times}$ は ω の成分から構成される歪対称行列で、つぎのように定義されます。

$$[\omega]_{\times} = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix} \quad (1.11)$$

クォータニオンの時間変化においても対応する関係式として

$$\dot{q} = \frac{1}{2}q\omega_q \quad (1.12)$$

が成り立ちます。ただし、 ω_q はスカラ部が 0、ベクトル部が ω の四元数です。右辺がクォータニオンの積で書かれていることに注意してください。

1.3.1 数値積分における正当性の修復・補正

シミュレーションで刻み時間を用いた繰り返し計算をすると、数値誤差が蓄積してしまいがちです。その対策として、回転行列 R の正規直交性を保つための補正処理、あるいはクォータニオン q の性質を保つための処理がプログラムに組み込む必要があります。

クォータニオン q の場合は「正規化」という処理で、計算結果のクォータニオンをノルム 1 に戻す処理を加えます。 q のノルムを求めて、それで q を割ることで、ノルム 1 のクォータニオンに戻すことができます。

回転行列 R の場合はさまざまな方法が考えられます。正規直交性を保つために行列の各列ベクトルを正規化する、あるいは角度と軸の情報に変換してから再度回転行列に戻す、などの方法があります。C 言語シミュレータでは後者の方法を用いています。